

1. 데이터의 기본 구성 요소

데이터는 우리가 정보를 저장하고 처리하는 가장 기본적인 단위입니다. 작은 단위들이 모여 더 큰 의미 있는 정보가 됩니다.

- **필드 (Field) / 아이템 (Item):** 데이터의 가장 작은 단위입니다. 예를 들어, 학생 정보에서 '이름', '학번', '점수' 등이 각각 필드입니다.
 - **워드 (Word):** 필드를 더 세분화한 단위입니다. '하프워드', '풀워드', '더블워드' 등으로 나뉩니다.
 - **문자 (Character):** 워드를 구성하는 가장 작은 단위입니다. 컴퓨터에서 문자를 표현하는 방식으로 크게 두 가지가 있습니다.
 - **아스키 코드 (ASCII):** 현재 가장 보편적으로 사용되는 7비트 또는 8비트 문자 인코딩 방식입니다. 주로 영문과 특수문자를 표현합니다. [Image of ASCII table]
 - **EBCDIC 코드 (Extended Binary Coded Decimal Interchange Code):** IBM 메인프레임 시스템에서 주로 사용되는 8비트 문자 인코딩 방식입니다.
 - 과거 4비트(니블) 시대에는 8421 코드, 2421 코드 등 다양한 코드가 사용되기도 했습니다.
- **레코드 (Record):** 여러 필드가 모여 하나의 완전한 정보를 나타내는 단위입니다. 예를 들어, 한 학생의 '이름', '학번', '점수' 필드가 모두 합쳐진 것이 하나의 레코드입니다. 레코드는 **정렬의 대상**이 됩니다.

2. 정렬의 핵심: 키 (Key)

데이터를 정렬하거나 특정 레코드를 찾기 위해서는 ****키(Key)****라는 기준점이 필요합니다.

- **기본키 (Primary Key):** 레코드들을 **유일하게 식별**할 수 있는 키입니다. **중복이 허용되지 않습니다.** 예를 들어, 학생 테이블에서 '학번'은 기본키가 될 수 있습니다. 같은 학번을 가진 학생은 없기 때문입니다.
- **후보키 (Candidate Key) / 외래키 (Foreign Key):** 강의에서는 중복을 허용하는 키로 설명되었지만, 일반적으로 **후보키**는 기본키가 될 수 있는 여러 키들을 의미하며, **외래키**는 다른 테이블의 기본키를 참조하여 관계를 맺는 키를 의미합니다. 예를 들어, 한 학생이 여러 과목을 수강할 때, 과목 성적 테이블에서 학생의 '학번'은 외래키가 될 수 있습니다.

3. 정렬 알고리즘의 중요성과 분류

모든 데이터는 정렬의 대상이 될 수 있으며, 효과적인 정렬은 데이터 처리의 효율성을 크게 높입니다.

- **정렬 알고리즘 선택 기준:**
 - **레코드 수와 크기:** 데이터의 양과 각 레코드의 크기에 따라 적합한 알고리즘이 달라집니다.
 - **키의 특성:** 키가 숫자인지, 문자인지, 실수인지 등 데이터 타입에 따라 효율적인 알고리즘이 다릅니다. (예: 구조체 안의 다양한 데이터 타입)
 - **정렬 위치:** 데이터를 어디에서 정렬하느냐에 따라 **내부 정렬**과 **외부 정렬**로 나뉩니다.

3.1. 내부 정렬 vs. 외부 정렬

- **내부 정렬 (Internal Sort):**
 - 모든 데이터가 컴퓨터의 **주기억장치(RAM)**에 저장된 상태에서 정렬하는 방식입니다.
 - 데이터 양이 많지 않을 때 효율적입니다.
 - **예시:** 엑셀 시트에서 몇백 개 안 되는 데이터를 정렬하는 경우.
 - [Image of RAM modules]

- **외부 정렬 (External Sort):**

- 대부분의 데이터가 하드디스크나 테이프와 같은 외부 기억장치에 저장되어 있고, 일부만 주기억 장치로 불러와 정렬하는 방식입니다.
- 데이터 양이 매우 많아 한 번에 메모리에 올릴 수 없을 때 사용합니다.
- **예시:** 수억 건의 고객 데이터를 정렬할 때, 데이터를 작은 블록으로 나누어 메모리에서 정렬하고 다시 외부 저장장치에 쓰는 과정을 반복하는 경우. (과거 테이프 장치에서 많이 사용)
- [Image of a tape drive]

3.2. 정렬 알고리즘의 평가와 안정성 🍷

모든 상황에 최적인 정렬 알고리즘은 없습니다. 각 알고리즘은 특정 상황에서 장단점을 가집니다.

- **평가 기준:**

- **비교 횟수:** 데이터를 몇 번 비교해야 하는가.
- **이동 횟수:** 데이터를 정렬하기 위해 몇 번 이동해야 하는가.

- **안정성 (Stability):**

- **동일한 키 값을 가진 레코드들의 상대적인 위치가 정렬 후에도 바뀌지 않는 것**을 의미합니다.
- 강의에서 언급된 전기요금과 TV 요금 예시처럼, 2500원이라는 동일한 요금이 있을 때 정렬 후에도 전기요금이 TV 요금보다 먼저 나왔다면 안정적인 정렬입니다.
- **안정적인 정렬:** 삽입 정렬, 버블 정렬
- **불안정한 정렬:** 선택 정렬

3.3. 주요 정렬 알고리즘 🍷

강의에서는 크게 두 가지 유형의 정렬 알고리즘을 소개했습니다.

(1) 단순하지만 비효율적인 정렬 ($O(N^2)$)

특징: 구현이 간단하지만, 데이터의 양이 늘어날수록 비교 횟수와 이동 횟수가 급격히 증가하여 효율이 떨어집니다.

- **선택 정렬 (Selection Sort):**

- **개념:** 정렬되지 않은 부분에서 **가장 작은(또는 큰) 값을 찾아 정렬된 부분의 올바른 위치로 교환**해 나가는 방식입니다. 마치 빈 그릇에 가장 작은 돌멩이를 하나씩 골라 담는 것과 같습니다.
- **동작 방식:**
 1. 정렬되지 않은 리스트에서 **최솟값**을 찾습니다.
 2. 찾은 최솟값을 정렬되지 않은 리스트의 첫 번째 요소(혹은 다음 정렬될 위치)와 교환합니다.
 3. 이 과정을 리스트 전체가 정렬될 때까지 반복합니다.
- **예시:** [5, 2, 8, 1, 9] 를 오름차순으로 정렬
 1. 리스트에서 가장 작은 값 1을 찾고, 5와 교환: [1, 2, 8, 5, 9]
 2. 나머지 [2, 8, 5, 9] 에서 가장 작은 값 2를 찾고, 2와 교환 (이미 제 위치): [1, 2, 8, 5, 9]
 3. 나머지 [8, 5, 9] 에서 가장 작은 값 5를 찾고, 8과 교환: [1, 2, 5, 8, 9]
 4. 나머지 [8, 9] 에서 가장 작은 값 8을 찾고, 8과 교환 (이미 제 위치): [1, 2, 5, 8, 9]
- **시간 복잡도:** $O(N^2)$ (비교 횟수 및 이동 횟수 모두)
- **안정성:** 불안정 (동일한 키 값의 상대적 위치가 바뀔 수 있음)

○

• 삽입 정렬 (Insertion Sort):

- **개념:** 이미 정렬된 부분에 ****새로운 요소를 적절한 위치에 '삽입'****하는 과정을 반복합니다. 마치 카드 게임에서 손에 든 카드들을 정렬하면서 새로운 카드를 받으면 제자리에 끼워 넣는 것과 같습니다.
- **동작 방식:**
 1. 두 번째 요소부터 시작하여, 현재 요소를 그 앞의 정렬된 부분에 있는 요소들과 비교하며 적절한 위치를 찾습니다.
 2. 적절한 위치를 찾으면 해당 위치에 삽입하기 위해 그 뒤의 요소들을 한 칸씩 뒤로 이동시킵니다.
 3. 이 과정을 모든 요소에 대해 반복합니다.
- **예시:** $[5, 2, 8, 1, 9]$ 를 오름차순으로 정렬
 1. $[5]$ (정렬됨)
 2. 2 를 5 와 비교하여 앞에 삽입: $[2, 5]$
 3. 8 을 $[2, 5]$ 와 비교하여 뒤에 삽입: $[2, 5, 8]$
 4. 1 을 $[2, 5, 8]$ 와 비교하여 가장 앞에 삽입: $[1, 2, 5, 8]$
 5. 9 를 $[1, 2, 5, 8]$ 와 비교하여 뒤에 삽입: $[1, 2, 5, 8, 9]$
- **시간 복잡도:**
 - **최선:** $O(N)$ (이미 정렬되어 있을 때)
 - **최악:** $O(N^2)$ (역순으로 정렬되어 있을 때)
- **특징:**
 - 레코드 이동이 많아 레코드 크기가 크면 불리합니다.
 - **안정적** (동일한 키 값의 상대적 위치가 유지됨)
 - 대부분 정렬되어 있는 경우 매우 효율적입니다.
-

• 버블 정렬 (Bubble Sort):

- **개념:** 인접한 두 요소를 비교하여 순서가 맞지 않으면 **서로 교환하는 과정을 반복**합니다. 마치 거품(bubble)이 물 위로 떠오르듯, 가장 큰(또는 작은) 값이 한 번의 스캔으로 맨 끝으로 이동하는 방식입니다.
- **동작 방식:**
 1. 첫 번째 요소부터 인접한 두 요소를 비교합니다.
 2. 순서가 맞지 않으면 교환합니다.
 3. 이 과정을 리스트의 끝까지 반복하면, 가장 큰(또는 작은) 값이 맨 끝으로 이동합니다.
 4. 정렬되지 않은 나머지 요소들에 대해 이 과정을 반복합니다.
- **예시:** $[5, 2, 8, 1, 9]$ 를 오름차순으로 정렬
 - 1회전:
 - $(5, 2)$ 비교 & 교환 -> $[2, 5, 8, 1, 9]$
 - $(5, 8)$ 비교 & 유지 -> $[2, 5, 8, 1, 9]$
 - $(8, 1)$ 비교 & 교환 -> $[2, 5, 1, 8, 9]$
 - $(8, 9)$ 비교 & 유지 -> $[2, 5, 1, 8, 9]$ (가장 큰 9가 맨 끝으로 이동)
 - 2회전: $[2, 5, 1, 8]$ 에 대해 반복...
- **시간 복잡도:** $O(N^2)$ (최선, 평균, 최악 모두 동일)
- **특징:**
 - 레코드 이동이 과다하여 비효율적입니다.

- **안정적** (동일한 키 값의 상대적 위치가 유지됨)

○

(2) 복잡하지만 효율적인 정렬 (빠른 정렬)

특징: 구현은 더 복잡하지만, 데이터의 양이 많을 때 훨씬 효율적입니다.

- **퀵 정렬 (Quick Sort):** 특정 기준점(피벗)을 중심으로 데이터를 분할하여 정렬하는 방식입니다.
- **힙 정렬 (Heap Sort):** 힙(Heap)이라는 자료구조를 활용하여 정렬하는 방식입니다.
- **병합 정렬 (Merge Sort):** 데이터를 작은 단위로 분할하고, 각각 정렬한 후 다시 병합하는 '분할 정복' 방식입니다.
- **기수 정렬 (Radix Sort):** 숫자의 자릿수를 기준으로 정렬하는 방식입니다.

4. 교수님께서 중요하다고 강조하신 부분 (시험 출제 예상)

강의 스크립트에서 교수님이 여러 번 강조하며 "시험에 나왔잖아", "중요하다", "실습은 이렇게 해야 한다"고 말씀하신 부분을 종합하면 다음과 같습니다.

- **키 (Key)의 개념과 종류 (기본키, 후보키/외래키):** 기본키가 '중복이 허용되지 않는 유일한 키'라는 정의를 명확히 이해해야 합니다.
 - "기본키라는 것은 어떤 키예요. 기본키라는 것은 중복이 허용되지 않는 유일한 키 유니크한 키 이것을 기본키라고 하잖아요. 중복을 허용하는 키 중복이 최소한 허용되는 키 이걸 후보키 후보키 폴링키 이렇게 이야기하잖아요."
- **문자 표현 방법 (아스키 코드, EBCDIC 코드):** 중간고사에 출제되었던 내용임을 언급하며 중요성을 강조 하셨습니다.
 - "문자 문자를 표현하는 방법은 두 가지가 있다고 이야기했죠. 어 코드 표현해서 중간고사에 나왔잖아. 어 문자 문자를 표현하는 방법은 크게 두 가지가 있다."
- **정렬 알고리즘의 평가 기준 (비교 횟수, 이동 횟수):** 효율성을 판단하는 핵심 지표입니다.
 - "정렬 알고리즘의 평가에서는 비교 횟수 몇 번 비교해서 정렬이 되는지를 비교해야 돼요. 하나하나씩 정렬 비교해서 정렬하는 방법이 있고 그다음에 임의의 구간을 나누어서 정렬하는 방법이 있고 ... 그 비교 비교하는 횟수 비교에 따른 이동하는 횟수 이와 같은 부분들이 중요하다 이런 부분들이"
- **각 정렬 알고리즘의 특징 (특히 단순하지만 비효율적인 정렬):**
 - **선택 정렬:** 단순하지만 비효율적인 이유 (비교/이동 횟수 많음), 시간 복잡도 $O(N^2)$, 안정성 불만족.
 - "선택 정렬은 이야기 했잖아요. 선택 초기에 단순하지만 비효율적인 방법이다. 왜 왜 단순하지만 비효율적이냐 단순하잖아요. ... 비교 횟수와 이동 횟수가 어떻게 돼요. 굉장히 많이 소요될 거 아니에요."
 - "전체적인 시간의 복잡도를 이야기하면은 각각 각각 비교해서 이동을 해줘야 되기 때문에 비교의 nn^2 승의 결과값을 가짐으로써 그리 안전성을 만족하지 않는다고 보는 게 있습니다."
 - **삽입 정렬:** 정렬 방식 (적절한 위치에 삽입), 안정적인 정렬 방법이라는 점, 이미 정렬되어 있으면 효율적이라는 점, 최악의 경우 $O(N^2)$ (역순일 때).
 - "삽입 정렬은 정렬되어 있는 부분에 새로운 레코드를 적절한 위치에 삽입하는 과정을 반복한다."
 - "안정된 정렬 방법이다. 왜 그러냐면은 그 정렬해 놓고 다시 그 위치를 조정해서 원하는 위치에 추가하기 때문에 삽입하기 때문에 안정된 정렬이라고 할 수 있고 이미 정렬되어 있으

면은 효율적이다."

- "최악의 경우에는 비교의 n^2 승의 결과값을 갖는다 라고 보면 되겠습니다."

- **버블 정렬:** 정렬 방식 (인접한 두 개 교환), 시간 복잡도 $O(N^2)$ (모든 경우 동일), 레코드 이동 과다.

- "버블 정렬이라는 것은 인접한 두 개의 레코드를 비교해서 순서대로 되어 있지 않으면 서로 교환되어 있다. 교환한다."
- "버블 정렬도 비교 횟수가 뭐니까? 최상위나 표준이나 최악일 경우에 모두 동일하죠. ... 비교의 n^2 승의 불과 값을 갖는 경우가 있죠. 이동 횟수도 이와 같이 비교의 n 의 2승으로 결과 값을 갖는다 따라서 레코드 이동에 있어서 레코드 이동이 과다하다"

- **정렬 알고리즘의 안정성 (Stability):** 개념을 정확히 이해하고, 각 정렬 알고리즘의 안정성 여부를 알아야 합니다. 동일한 키 값의 상대적 위치가 정렬 후에도 바뀌는지 여부가 핵심입니다.

- "안정성이라는 것은 동일한 키값을 받은 레코드들의 상대적 위치가 정렬 후에도 바뀌지 않는 거예요."
- "이런 경우는 안정하지 않는 경우다라고 할 수 있죠. 바뀌어버렸다는 것은 무엇이다. 안정하지 않는다는 의미라고 봅니다."

- **프로그래밍 실습: main 함수 수정 및 단계별 출력:** 교수님이 각 정렬 알고리즘의 main 함수를 교재와 다르게 수정하여, 정렬 과정(단계별 비교, 이동 횟수)이 출력되도록 실습 과제를 명확히 제시했습니다. 이 부분이 가장 강조된 부분 중 하나입니다.

- "책만 보고 짜는 게 아니라 지금 현재 우리 이제 그 개인에서 랜덤으로 데이터를 생성해 가지고 했는데 이렇게 리스트를 이용해서 6개의 값을 줘가지고 ... 이와 같은 형태로 출력하는 형태로 실습을 하시길 바랍니다."
- "실습은 교재의 교재에 95%의 하 그대로 좀 돼야 단 이런 부분들이 나오게 하려면은 이제 여기에서 프린트해 그다음에 여기에서 각각 단계별로 printf를 써서 몇 번 6개니까 5번 서로 비교 등간 비교하면은 이렇게 정렬이 되잖아요. 그래서 몇 단계로 나눠서 정렬이 되는지 이렇게 바꾸어서 프로그램을 작성해서 업로드하시면 되겠습니다."
- "우리 교재에 몇 쪽을 보시면 되냐면은 449쪽 448쪽에 선택 정렬 함수가 나와 있고 449쪽에 보시면은 이제 이것에 대한 이제 전체적인 문제 프로그램이 나와 있는데, 이렇게 출력되는 형태의 실습을 하시는 게 아니라 형태를 메인을 이렇게 바꾸어 가지고 바꿔서 여러분들이 프로그램을 완성하시면 되겠습니다."